Universidade Católica de Pernambuco (UNICAP)

Disciplina: Análise de Algoritmos | Lista de Exercícios – 2º GQ

Professor: Pedro Ximenes **Semestre:** 2026.2

Assuntos: Classes P e NP, Certificados e Verificadores, Reduções Polinomiais, NP-Completude e Decidibilidade

---

Instruções: Resolva as questões de forma clara e objetiva. Nos itens de Verdadeiro/Falso, marque **V** ou **F** e **justifique** em 1–2 linhas (um item sem justificativa não vale ponto). Nas demonstrações, deixe explícito o certificado, o verificador e os custos assintóticos envolvidos. As definições dos problemas citados estão logo abaixo.

Definições dos problemas

Problema: SAT
Entrada: uma fórmula $\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$ em forma normal conjuntiva sobre as variáveis $x_1, \ldots, x_n$, em que cada cláusula C_i é uma disjunção de literais (uma variável x_j ou sua negação $\neg x_j$).
Pergunta: existe uma atribuição de valores verdade às variáveis que torna φ verdadeira?

Problema: CLIQUE
Entrada: um grafo $G = (V, E)$ e um inteiro k.
Pergunta: existe $C \subseteq V$ com $|C| \geq k$ tal que todo par de vértices distintos de C é adjacente em G?

Problema: CONJUNTO INDEPENDENTE (IND-SET)
Entrada: um grafo $G = (V, E)$ e um inteiro k.
Pergunta: existe $S \subseteq V$ com $|S| \geq k$ tal que nenhum par de vértices de S é adjacente em G?

Problema: SUBSET-SUM
Entrada: um conjunto (ou multiconjunto) S de inteiros e um inteiro-alvo k.
Pergunta: existe $T \subseteq S$ tal que $\sum_{t \in T} t = k$?

Problema: 3-COLORAÇÃO
Entrada: um grafo $G = (V, E)$.
Pergunta: existe uma função $c : V \to \{1, 2, 3\}$ tal que $c(u) \neq c(v)$ para toda aresta $uv \in E$?

Problema: HALT (Problema da Parada)
Entrada: o código de um programa P e uma entrada x.
Pergunta: a execução de P sobre x termina?

Notação: $A \leq_p B$ indica que A se reduz a B em tempo polinomial; $\bar{G}$ denota o grafo complementar de G (mesmos vértices; $uv \in E(\bar{G}) \iff uv \notin E(G)$, para $u \neq v$).

Parte I – Conceitos Fundamentais

---

1. **Vocabulário da complexidade.** Defina formalmente:
   (a) algoritmo eficiente;
   (b) problema de decisão e problema de otimização (dê um exemplo de cada, relacionados entre si);
   (c) certificado positivo e algoritmo verificador;
   (d) as classes P, NP, NP-completo e NP-difícil.

2. **NP e NP-completo.**
   (a) Defina formalmente a classe NP, mencionando certificado, verificador e as restrições de tamanho e de tempo de cada um.
   (b) Defina formalmente a classe dos problemas NP-completos (as duas condições).
   (c) Informalmente, o que significa dizer que um problema é NP-completo? Por que essa informação é útil para quem precisa implementar uma solução para ele?
   (d) Explique a diferença entre um problema NP-completo e um problema NP-difícil. Dê um exemplo de problema NP-difícil que *não* é de decisão.

3. **Verificar vs. resolver (V/F).** Julgue e justifique:

(a) Todo problema pertencente a NP pode ser resolvido em tempo polinomial.

(b) $P \subseteq NP$, pois todo problema resolvível em tempo polinomial também possui verificador polinomial.

(c) Um certificado válido para NP pode ter tamanho exponencial em $|x|$, desde que o verificador seja polinomial.

(d) Se $L \in P$, então o próprio algoritmo de decisão de L serve como verificador, ignorando o certificado.

(e) Um verificador determinístico polinomial é suficiente para *resolver* o problema sem receber um certificado.

(f) "NP" significa "não polinomial".

4. **Certificados na prática (SUBSET-SUM).** Considere a instância $S = \{8, 3, 11, 5, 2, 6\}$ e $k = 16$.

   (a) Para cada candidato abaixo, execute o verificador passo a passo (pertinência a S e soma) e diga se ele é ou não um certificado válido:

   $$T_1 = \{8, 6, 2\}, \qquad T_2 = \{3, 11, 5\}, \qquad T_3 = \{8, 7, 1\}, \qquad T_4 = \{11, 5\}.$$

   (b) A instância é SIM ou NÃO? O fato de T_2 ser inválido permite concluir que a instância é NÃO? Explique.

   (c) Quantos subconjuntos um algoritmo de força bruta precisaria testar, no pior caso, para uma instância com $|S| = n$? Compare com o custo de *verificar* um único candidato.

Parte II – Mostrando que um problema está em NP

5. **SAT $\in NP$.**

   (a) Mostre que SAT está em NP: descreva o certificado, o algoritmo verificador e o custo do verificador em função de n (variáveis) e do tamanho total da fórmula.

   (b) Considere $\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2) \wedge (\neg x_3 \vee \neg x_2) \wedge (x_1 \vee x_3)$. Execute seu verificador para as atribuições

   $$\text{(i) } x_1 = 1,\ x_2 = 1,\ x_3 = 0 \qquad \text{(ii) } x_1 = 1,\ x_2 = 0,\ x_3 = 1$$

   indicando, para cada cláusula, se ela é satisfeita. O que se conclui sobre φ?

6. **3-COLORAÇÃO $\in NP$.**

   (a) Mostre que o problema da 3-coloração está em NP (certificado, verificador e custo).

   (b) Seja G com $V = \{a, b, c, d, e\}$ e $E = \{ab, bc, cd, de, ea, ac\}$. Verifique se $c(a) = 1$, $c(b) = 2$, $c(c) = 3$, $c(d) = 1$, $c(e) = 2$ é uma 3-coloração válida, conferindo aresta por aresta.

   (c) Explique por que o grafo completo K_4 não admite 3-coloração. Isso exige testar todas as 3^4 atribuições?

7. **SUBSET-SUM $\in NP$ (formal).** Usando a definição formal de NP, prove que SUBSET-SUM pertence a NP. Descreva o certificado, mostre que seu tamanho é polinomial no tamanho da entrada, escreva o verificador em pseudocódigo e analise seu custo assintótico.

8. **CLIQUE e CONJUNTO INDEPENDENTE em NP.**

   (a) Mostre que CLIQUE $\in NP$. Quantos pares precisam ser verificados para um candidato C com $|C| = k$? Esse custo é polinomial em quê?

   (b) Mostre que CONJUNTO INDEPENDENTE $\in NP$.

   (c) Suponha que o grafo seja dado por *lista* de adjacência em vez de matriz. O custo do verificador continua polinomial? Justifique.

Parte III – Reduções Polinomiais e NP-Completude

9. **Direção da redução.** É verdade que, se reduzirmos um problema em P para um problema em NP, então $P = NP$? Justifique, explicando o que uma redução $A \leq_p B$ permite concluir sobre A e sobre B.

10. **O que se conclui de uma redução?** Seja A um problema NP-completo e B um problema qualquer.

    (a) Diga tudo o que podemos concluir ao mostrar $A \leq_p B$.

    (b) Diga tudo o que podemos concluir ao mostrar $B \leq_p A$.

    (c) Em qual dos dois casos, acrescentando a hipótese $B \in NP$, podemos concluir que B é NP-completo?

11. **Redução e cotas de tempo.** Sejam A e B dois problemas na classe P. Verdadeiro ou falso: *se existe algoritmo com tempo $\Theta(n^2)$ que resolve B e A pode ser reduzido para B em tempo polinomial, então não existe algoritmo com tempo $\Theta(n)$ que resolve A.* Justifique (sugestão: pense em um exemplo concreto de A simples reduzido a um B mais caro).

12. **Reduções (V/F).** Julgue e justifique:
    (a) Um problema Π é NP-completo se: (i) $\Pi \in NP$ e (ii) todo $L \in NP$ satisfaz $L \leq_p \Pi$.
    (b) Se L_1 é NP-completo e $L_1 \leq_p L_2$, então L_2 é NP-completo.
    (c) $A \leq_p B$ significa que B é pelo menos tão difícil quanto A.
    (d) Se $A \leq_p B$ e $A \in P$, então $B \in P$.
    (e) Se $A \leq_p B$ e $B \in P$, então $A \in P$.
    (f) Redução polinomial é transitiva: se $L_1 \leq_p L_2$ e $L_2 \leq_p L_3$, então $L_1 \leq_p L_3$.
    (g) Se L_1 é NP-completo, $L_2 \in NP$ e $L_2 \leq_p L_1$, então L_2 é NP-completo.
    (h) A redução NOVO-PROBLEMA $\leq_p$ SAT prova que NOVO-PROBLEMA é NP-completo.

13. **CLIQUE $\leq_p$ CONJUNTO INDEPENDENTE.** Prove que CONJUNTO INDEPENDENTE é NP-completo usando a redução $f(G, k) = (\bar{G}, k)$.
    (a) Mostre que CONJUNTO INDEPENDENTE $\in NP$ (pode citar a Questão 8).
    (b) Descreva formalmente f e prove que ela é computável em tempo polinomial, indicando o custo de construir $\bar{G}$ a partir de uma matriz de adjacência.
    (c) Prove a equivalência nos dois sentidos:
    ($\Rightarrow$) se G possui clique de tamanho k, então $\bar{G}$ possui conjunto independente de tamanho k;
    ($\Leftarrow$) se $\bar{G}$ possui conjunto independente de tamanho k, então G possui clique de tamanho k.
    (d) Explique por que a direção da redução é CLIQUE $\leq_p$ IND-SET (e não o contrário) para concluir que IND-SET é NP-difícil.

14. **A redução em uma instância concreta.** Seja G com $V = \{1, 2, 3, 4, 5\}$ e $E = \{12, 13, 23, 24, 34, 45\}$.
    (a) Liste as arestas de $\bar{G}$.
    (b) Encontre uma clique de tamanho 3 em G e verifique que ela é um conjunto independente em $\bar{G}$.
    (c) G possui clique de tamanho 4? Use $\bar{G}$ para justificar sua resposta.
    (d) A mesma função $f(G, k) = (\bar{G}, k)$ também reduz IND-SET a CLIQUE? Justifique usando o fato $\bar{\bar{G}} = G$.

15. **Provando NP-completude: o roteiro.** Um colega quer provar que o problema Π' é NP-completo e apresenta apenas uma redução SUBSET-SUM $\leq_p \Pi'$.
    (a) O que falta na prova dele? O que ele já pode concluir com o que apresentou?
    (b) Escreva o roteiro completo (em passos) para provar que um problema novo Π' é NP-completo.
    (c) Se, por engano, ele tivesse mostrado $\Pi' \leq_p$ SUBSET-SUM, o que isso provaria?

16. **Cook-Levin e as consequências de $P = NP$ (V/F).** Julgue e justifique:
    (a) Pelo Teorema de Cook-Levin, SAT foi o *primeiro* problema provado NP-completo; todo problema em NP pode ser reduzido a SAT em tempo polinomial.
    (b) Se qualquer problema NP-completo for resolvido em tempo polinomial, então $P = NP$.
    (c) Se for provado que um problema NP-completo específico não admite algoritmo polinomial, então $P \neq NP$.
    (d) Se $P = NP$, todo problema NP-completo passaria a ter algoritmo polinomial.
    (e) Hoje já se sabe que $P \neq NP$.

Parte IV – Decidibilidade e Limites da Computação

17. **Decidibilidade (V/F).** Julgue e justifique:
    (a) SAT é NP-completo, porém *decidível*: existe algoritmo (exponencial) que o resolve corretamente em todos os casos.
    (b) Todo problema NP-completo é também indecidível.

(c) CLIQUE é decidível: a força bruta testa todos os subconjuntos de tamanho k e sempre termina com a resposta correta.

(d) Se um programa $P(x)$ não terminou após 10^9 passos, podemos concluir que $P(x)$ entra em loop infinito.

(e) Existe um algoritmo que, dados o código de qualquer programa P e uma entrada x, determina corretamente em todos os casos se $P(x)$ termina.

(f) HALT é indecidível por limitação dos computadores atuais; computadores quânticos ou mais rápidos poderiam decidi-lo.

18. **A indecidibilidade de HALT.** Reconstrua a prova por contradição (diagonalização) de que HALT é indecidível:

(a) Suponha que existe um decisor $H(P, x)$. O que H devolve e quais propriedades ele precisa ter?

(b) Construa o programa $D(P)$ que usa H e escreva-o em pseudocódigo.

(c) Analise a execução de $D(D)$ nos dois casos possíveis e mostre que ambos levam a uma contradição.

(d) Por que essa prova vale para *qualquer* linguagem de programação ou hardware?

19. **Decidível $\neq$ tratável.**

(a) Explique a diferença entre um problema *decidível*, um problema *tratável* (em P) e um problema *indecidível*.

(b) Classifique cada problema como: (i) em P; (ii) NP-completo; (iii) indecidível. Justifique brevemente cada um.

- IS-SORTED: dado um array $A[0..n-1]$, ele está em ordem não-decrescente?
- Existe caminho de a até b em um grafo G?
- Existe árvore geradora de peso no máximo k em um grafo ponderado?
- SAT.
- CLIQUE.
- HALT.

(c) Desenhe um diagrama (conjuntos) relacionando decidíveis, NP, P e NP-completos, assumindo $P \neq NP$. Onde fica HALT?

Parte V – Questões de Concursos (Enade e Poscomp)

Para cada questão, assinale a alternativa correta e justifique por que as demais estão incorretas.

20. **(Enade)** Considere o processo de fabricação de um produto siderúrgico que necessita passar por n tratamentos térmicos e químicos para ficar pronto. Cada uma das n etapas de tratamento é realizada uma única vez na mesma caldeira. Além do custo próprio de cada etapa do tratamento, existe o custo de se passar de uma etapa para outra, uma vez que, dependendo da sequência escolhida, pode ser necessário alterar a temperatura da caldeira e limpá-la para evitar a reação entre os produtos químicos utilizados. Assuma que o processo de fabricação inicia e termina com a caldeira limpa. Deseja-se projetar um algoritmo para indicar a sequência de tratamentos que possibilite fabricar o produto com o menor custo total. Nessa situação, conclui-se que

(A) a solução do problema é obtida em tempo de ordem $O(n \log n)$, utilizando-se um algoritmo ótimo de ordenação.

(B) uma heurística para a solução do problema de coloração de grafos solucionará o problema em tempo polinomial.

(C) o problema se reduz a encontrar a árvore geradora mínima para o conjunto de etapas do processo, requerendo tempo de ordem polinomial para ser solucionado.

(D) a utilização do algoritmo de Dijkstra para se determinar o caminho de custo mínimo entre o estado inicial e o final soluciona o problema em tempo polinomial.

(E) qualquer algoritmo conhecido para a solução do problema descrito possui ordem de complexidade de tempo não-polinomial, uma vez que o problema do caixeiro viajante se reduz a ele.

21. **(Enade)** Uma fazenda possui um único poço artesiano que deve abastecer n bebedouros para o gado. Deseja-se determinar um projeto de ligação entre esses $n + 1$ pontos através de encanamentos com a menor extensão total. Um algoritmo proposto para a solução do problema executa os seguintes passos:

1. Crie $n + 1$ conjuntos unitários, cada um contendo um dos pontos a serem ligados entre si, e insira esses conjuntos em um conjunto C.
2. Crie um conjunto D contendo um registro para cada combinação possível de dois pontos distintos a serem ligados. Cada registro deve conter os campos c_i, c_j e d, em que c_i e c_j são os dois pontos a serem ligados e d é a distância entre eles.
3. Enquanto D não estiver vazio, faça:

3.1. Remova o registro de D com o menor valor de distância d.
3.2. Se os valores de c_i e c_j do registro removido pertencerem a conjuntos distintos de C, então:
3.2.1. Substitua estes dois conjuntos pela união entre eles.
3.2.2. Guarde o registro removido em um conjunto-solução.

Com base na descrição do problema e do algoritmo proposto, conclui-se que

(A) o problema exemplifica a obtenção de uma árvore geradora mínima, portanto está no conjunto P.

(B) o algoritmo é uma heurística para o Problema do Caixeiro Viajante, logo apresenta complexidade polinomial.

(C) o problema descrito é de otimização, logo pertence ao conjunto NP-difícil, mas não ao conjunto NP-completo.

(D) uma alternativa para a solução do problema é usar o algoritmo de Dijkstra para obtenção do caminho mínimo entre dois pontos.

(E) o passo de maior custo do algoritmo é a criação do conjunto D com as combinações de pontos, apresentando complexidade computacional $O(n!)$.

22. **(Enade)** Um cientista afirma ter encontrado uma redução polinomial de um problema NP-Completo para um problema pertencente à classe P. Considerando que esta afirmação tem implicações importantes no que diz respeito à complexidade computacional, avalie as seguintes asserções e a relação proposta entre elas.

I. A descoberta do cientista implica $P = NP$.

PORQUE

II. A descoberta do cientista implica na existência de algoritmos polinomiais para todos os problemas NP-Completos.

A respeito dessas asserções, assinale a opção correta.

(A) As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.

(B) As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.

(C) A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.

(D) A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.

(E) As asserções I e II são proposições falsas.

23. **(Poscomp)** Considere as seguintes afirmações sobre classes de problemas:

I. O problema de decisão CAM, descrito a seguir, pertence à classe de complexidade P.
CAM (caminho em grafo). Entrada: uma tripla (G, a, b) em que G é um grafo e a e b são nós de G. *Pergunta:* existe caminho em G iniciando em a e terminando em b?

II. Um problema X pertence à classe de problemas NP-completos quando satisfaz às seguintes condições: X pertence à classe NP, e todo problema Y da classe NP pode ser reduzido em tempo polinomial a X.

III. Se um problema de decisão X pertence à classe P, então o complemento do problema X (problema com as mesmas instâncias que X, porém com as respectivas respostas invertidas) pertence à classe NP.

Quais estão corretas?

(A) Apenas I.

(B) Apenas III.

(C) Apenas I e II.

(D) Apenas II e III.

(E) I, II e III.

24. **(Estilo Enade) Otimização vs. decisão.** Considere o Caixeiro Viajante (TSP): dado um grafo completo com custos nas arestas, encontrar um ciclo hamiltoniano de custo mínimo. Na versão de decisão (TSP_k), dado também um valor k, pergunta-se se existe ciclo hamiltoniano de custo no máximo k. Avalie as afirmações:

I. TSP_k pertence a NP, pois dado um ciclo é possível verificar em tempo polinomial que ele é hamiltoniano e tem custo $\leq k$.

II. TSP (otimização) não pertence a NP no sentido estrito, pois NP é uma classe de problemas de decisão; porém ele é NP-difícil.

III. Se TSP_k puder ser resolvido em tempo polinomial, então o custo ótimo do TSP também pode ser obtido em tempo polinomial (por exemplo, por busca binária sobre k, quando os custos são inteiros).

IV. Como o TSP é NP-difícil, é impossível resolvê-lo de forma exata para qualquer instância.

Estão corretas apenas: (A) I e II. (B) I, II e III. (C) II e IV. (D) I, III e IV. (E) I, II, III e IV.

Parte VI – Questão Discursiva

25. **NP-completude na engenharia de software.** Um gerente pede à sua equipe um algoritmo *exato e rápido* para alocar n tarefas a servidores minimizando o tempo total, e a equipe descobre que o problema é NP-difícil (por exemplo, por redução a partir de SUBSET-SUM).

    (a) Explique, em até 5 linhas, por que demonstrar a NP-completude/NP-dificuldade é relevante na prática, e como você justificaria ao gerente a impossibilidade (conhecida) de atender ao pedido.

    (b) Cite pelo menos três alternativas práticas que a equipe pode adotar (por exemplo: heurísticas, algoritmos aproximados, restringir a entrada a casos especiais, algoritmos exponenciais para n pequeno) e comente um prós/contras de cada.

Bons estudos!